RAG, Memory & Data Authorization

Enterprise Policy Interceptor Architecture for Agentic AI

Retrieval Augmented Generation (RAG) systems retrieve documents from a knowledge store and inject them into the LLM context. Without authorization controls at the retrieval layer, an agent can be manipulated into retrieving and exposing documents that the invoking user is not permitted to see. This is one of the most critical data security risks in enterprise AI.

✕ ANTI-PATTERN

Critical Risk: Without RAG authorization, a user in Sales can prompt an agent to retrieve confidential M&A; documents from the knowledge base simply by phrasing their query to match those documents. The vector similarity search has no concept of permissions — authorization must be added as a filter layer on top.

1.1 RAG Authorization Pipeline

```
USER QUERY (with canonical claims + context) ■
■■■■■■▼■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ P1: Retrieval Authorization ■■■■
Cedar: Can user query this knowledge base?
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ ALLOW
■■■■■■▼■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ Claims → Permission Set Mapping ■
■ { clearance_level, allowed_categories, ■ ■ tenant_id, department, geography } ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■
■■■■■■▼■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ PRE-RETRIEVAL FILTER ■ ■ (Metadata
filter applied to vector query)■ ■ WHERE tenant_id = :tenant ■ ■ AND classification IN
(:allowed_classes)■ ■ AND geography IN (:allowed_geos) ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■
■■■■■■▼■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ VECTOR SIMILARITY SEARCH ■ ■
(OpenSearch / pgvector / Bedrock KB) ■ ■ Filtered result set — only permitted docs■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■
■■■■■■▼■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ POST-RETRIEVAL AUTHORIZATION ■■■■
Cedar: Verify each retrieved chunk ■ (Per-document Cedar evaluation) ■ against user clearance
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ (filtered, authorized chunks only)
■■■■■■▼■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ CONTEXT INJECTION → LLM ■ ■ (Only
authorized content in context) ■ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■
■■■■■■▼■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ OUTPUT CLASSIFICATION FILTER ■■■■
Cedar: Is output within user's clearance? ■ (Scan response for leaked PII/classified)■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ AUTHORIZED RESPONSE (with source
citations)
```

1.2 Document-Level Authorization Schema

Every document and chunk in the knowledge store must carry authorization metadata. This metadata is the basis for both pre-retrieval filtering and post-retrieval Cedar policy evaluation:

```
{ "doc_id": "doc-m-and-a-briefing-2025-001", "title": "Project Phoenix M&A; Briefing",
"classification": "TOP_SECRET", "classification_basis": "MERGERS_ACQUISITIONS", "tenant_id":
"bank-prod", "legal_entity": "BANK_UK_LTD", "department_owners": ["STRATEGY", "LEGAL",
"C_SUITE"], "geography_restriction": ["GB", "IE"], "required_capabilities": [
"can_access_ma_documents", "can_view_strategic_plans" ], "need_to_know_list": ["emp-00001",
"emp-00042", "emp-00891"], "embargo_until": "2025-09-01T00:00:00Z", "retention_class":
"LEGAL_HOLD", "dlp_category": "CONFIDENTIAL_BUSINESS", "created_at": "2025-03-15T14:00:00Z",
"source_system": "SharePoint_Legal", "chunks": [ { "chunk_id": "chunk-001", "classification":
"TOP_SECRET", // Chunk may differ from doc "content_hash": "sha256:abc123..." } ] }
```

1.3 Cedar Policies for RAG Authorization

Classification: CONFIDENTIAL — INTERNAL USE ONLY

Published: June 2026 · AWS Well-Architected Series

```
// Pre-retrieval: can agent query this knowledge base? permit( principal is BankAI::Agent,
action == BankAI::Action::"QueryKnowledgeBase", resource is BankAI::KnowledgeBase ) when {
principal.delegatedFrom.capabilities.contains("can_query_knowledge_base") && resource.tenantId
== principal.tenantId && context.riskScore < 60 }; // Post-retrieval chunk authorization:
verify each chunk permit( principal is BankAI::Agent, action == BankAI::Action::"AccessChunk",
resource is BankAI::DocumentChunk ) when { // Tenant isolation — mandatory resource.tenantId ==
principal.tenantId && // Classification-based access principal.delegatedFrom.clearanceLevel >=
resource.classification && // Department owner check resource.departmentOwners.containsAny(
principal.delegatedFrom.department ) || // OR explicit need-to-know
resource.needToKnowList.contains(principal.delegatedFrom.id) }; // Embargo enforcement forbid(
principal, action == BankAI::Action::"AccessChunk", resource is BankAI::DocumentChunk ) when {
resource has embargoUntil && context.currentTime < resource.embargoUntil }; // Tenant isolation
— hard block (always evaluated) forbid( principal, action, resource is BankAI::DocumentChunk )
when { resource.tenantId != principal.tenantId };
```

1.4 Vector Database Filtering Implementation

Modern vector databases support metadata filtering at query time. The pre-retrieval filter must be constructed from the canonical claims BEFORE the similarity search is executed:

```
# Python: Construct pre-retrieval filter from canonical claims def
build_rag_filter(canonical_claims: dict) -> dict: user = canonical_claims['principal'] org =
canonical_claims['organization'] caps = canonical_claims['capabilities'] # Map clearance level
to permitted classifications clearance_map = { "L1": ["PUBLIC"], "L2": ["PUBLIC", "INTERNAL"],
"L3": ["PUBLIC", "INTERNAL", "CONFIDENTIAL"], "L4": ["PUBLIC", "INTERNAL", "CONFIDENTIAL",
"SECRET"], "L5": ["PUBLIC", "INTERNAL", "CONFIDENTIAL", "SECRET", "TOP_SECRET"], } clearance =
user.get('clearance_level', 'L2') allowed_classifications = clearance_map.get(clearance,
["PUBLIC"]) # Build OpenSearch / pgvector filter filter_query = { "bool": { "must": [ {"term":
{"tenant_id": org['tenant_id']}}, {"terms": {"classification": allowed_classifications}},
{"terms": {"geography_restriction": [org['geography'], "GLOBAL"]}}, ], "should": [ # Department
owner match OR need-to-know list {"terms": {"department_owners": [org['department']]}},
{"term": {"need_to_know_list": user['id']}}, ], "minimum_should_match": 1, "must_not": [ #
Embargo filter {"range": {"embargo_until": {"gte": "now"}}} ] } } return filter_query
```

2. Memory Authorization Architecture

AI agents use multiple types of memory. Each type has different authorization requirements. An agent must never access another user's memory without explicit authorization.

2.1 Memory Type Taxonomy and Authorization Model

Memory Type	Scope	Authorization Model	Cedar Policy
Working Memory (In-context)	Current conversation window	Owned by session — no cross-session access. Agent may only hold context from its own authorized sources.	Implicit — scope is session-bound
Episodic Memory (Short-term)	Recent interactions for this user-agent pair	Private to user. Agent can read own episodic memory only. Manager MAY have read access with explicit capability.	principal.userId == memory.ownerId
Long-term Memory (Semantic)	Learned patterns, user preferences	User-scoped. Cannot be shared without explicit consent. Tenant-isolated.	memory.tenantId == principal.tenantId AND memory.ownerId == principal.userId
Shared Memory (Team/Project)	Project context shared across users	Shared within a defined group. Group membership checked via Cedar. No cross-group access.	principal.projectMemberships contains memory.projectId
Organizational Memory (Enterprise KB)	Company-wide knowledge base	Full RAG authorization applies. Classification + capability-based.	Full RAG policy stack

2.2 Cedar Policies for Memory Protection

```
// Agents can only read memory they own (or are delegated to read) permit( principal is
BankAI::Agent, action == BankAI::Action::"ReadMemory", resource is BankAI::MemoryRecord ) when
{ // Own user's memory (resource.ownerId == principal.delegatedFrom.id && resource.tenantId ==
principal.tenantId) || // Manager access (with explicit capability)
(principal.delegatedFrom.capabilities.contains("can_read_team_memory") && resource.teamId ==
principal.delegatedFrom.teamId) }; // Memory write: agents write only to own user's memory
scope permit( principal is BankAI::Agent, action == BankAI::Action::"WriteMemory", resource is
BankAI::MemoryRecord ) when { resource.ownerId == principal.delegatedFrom.id &&
resource.tenantId == principal.tenantId && context.memoryScope in ["WORKING", "EPISODIC",
"PERSONAL_SEMANTIC"] }; // Shared project memory — read requires project membership permit(
principal is BankAI::Agent, action == BankAI::Action::"ReadMemory", resource is
BankAI::MemoryRecord ) when { resource.memoryType == "SHARED_PROJECT" &&
principal.delegatedFrom.projectMemberships.contains(resource.projectId) && resource.tenantId ==
principal.tenantId }; // Hard block: cross-tenant memory access is always forbidden forbid(
principal, action, resource is BankAI::MemoryRecord ) when { resource.tenantId !=
principal.tenantId };
```

2.3 Memory Protection Implementation

Memory Storage	Authorization Control	Implementation
DynamoDB (working/episodic)	Partition key = userId#tenantId; IAM policy restricts agent role to own partition	DynamoDB condition expressions + Cedar post-read verify

Memory Storage	Authorization Control	Implementation
OpenSearch (semantic memory)	Index-per-tenant pattern; OpenSearch security plugin (row-level security)	Pre-query filter + Cedar chunk authorization
Redis ElastiCache (working context)	Key prefix = tenant:user:session; no cross-key access in agent role	Redis AUTH + namespace isolation
S3 (long-term memory snapshots)	S3 object key = tenant/user-id/memory/; IAM boundary restricts agent	S3 resource policy + Cedar evaluation on read
RDS PostgreSQL (shared memory)	Row-level security policies (Postgres RLS) + Cedar post-read filter	Postgres RLS + Cedar authorization

3. Multi-Tenant Data Isolation

Multi-tenant AI deployments must guarantee that one tenant's agents, tools, knowledge, and memory can never interact with another tenant's data. This requires defense-in-depth: tenant isolation at the IAM layer, the data layer, and the Cedar policy layer.

3.1 Tenant Isolation Defense Layers

Layer	Isolation Mechanism	Enforced By
IAM / AWS Identity	ECS task roles scoped to tenant-specific resources	IAM policies, resource tags
Network	VPC per tenant or subnet isolation with NACLs	VPC, Security Groups, NACLs
Data Storage	Partition-per-tenant (DynamoDB, OpenSearch index, S3 prefix)	Storage configuration + IAM
Cedar Policy	Mandatory forbid: resource.tenantId != principal.tenantId	Cedar AVP (always evaluated)
Vector Search	Pre-query metadata filter: tenant_id = :tenant	RAG retrieval layer
Memory	Key namespace: tenant:userId — no cross-namespace access	Storage key design + IAM
API Gateway	Custom domain per tenant with tenant claim validation	API GW + Lambda Authorizer
Audit Logs	CloudTrail + per-tenant log group isolation	CloudWatch Log Groups
Encryption	Per-tenant KMS keys for data at rest	AWS KMS, CMK per tenant

3.2 Tenant Isolation Cedar Pattern

```
// TENANT ISOLATION: This policy MUST be the first evaluated. // It uses Cedar's forbid-unless pattern to guarantee isolation. // Even if another permit policy would allow access, this forbid // overrides it when tenant IDs do not match. // Forbid ALL cross-tenant access to any resource forbid( principal, action, resource ) when { // Resource has a tenant identifier resource has tenantId && // Principal has a tenant identifier principal has tenantId && // They do not match resource.tenantId != principal.tenantId }; // For agents: check the delegated user's tenant too forbid( principal is BankAI::Agent, action, resource ) when { resource has tenantId && principal.delegatedFrom has tenantId && resource.tenantId != principal.delegatedFrom.tenantId }; // Cedar evaluation order note: // Cedar evaluates ALL applicable policies. // A single forbid overrides ALL permits. // These tenant isolation forbids will always win.
```

4. Output Classification and Filtering

Authorization does not end when a tool executes or a document is retrieved. The output must be classified and filtered before being returned to the user or injected into the next agent step. This prevents data leakage through the LLM generation layer.

4.1 Output Classification Pipeline

```
AGENT OUTPUT (raw LLM response) ■ ■■■■■■▼■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■
AUTOMATED CLASSIFICATION ■ ■ • AWS Macie patterns (PII detection) ■ ■ • Bedrock Guardrails
(content policy) ■ ■ • Custom classifier (CONFIDENTIAL/SECRET)■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ classification = {PII: false,
class: INTERNAL} ■■■■■■▼■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ CEDAR POLICY
EVALUATION ■ ■ Can this user receive output of this ■ ■ classification from this agent? ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ ALLOW / DENY with redaction
obligations ■■■■■■▼■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ REDACTION / FILTERING
(if obligated) ■ ■ • PII: replace with [REDACTED] ■ ■ • Over-clearance: strip classified refs
■ ■ • Tenant data: remove cross-tenant refs ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■
■■■■■■▼■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ DLP SCAN (final gate) ■ ■ Amazon
Macie / custom patterns ■ ■ Block if DLP detects sensitive patterns ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■
■■■■■■▼■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ AUDIT LOG (output classification +
hash) ■ ■ CloudTrail + DynamoDB audit record ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ FILTERED, CLASSIFIED RESPONSE →
USER
```

4.2 Data Sensitivity Classification Model

Level	Label	Examples	Agent Policy
L0	PUBLIC	Marketing docs, public web content	All agents may retrieve and return
L1	INTERNAL	Internal memos, process docs	Authenticated users only, no external channels
L2	CONFIDENTIAL	Customer data, contracts, budgets	Capability required, MFA, logged
L3	SECRET	M&A; details, legal strategy, key material	Need-to-know list, MFA, approval, DLP active
L4	TOP_SECRET	Board-level strategy, regulator confidential	Explicit person list only, dual approval, offline audit

✓ BEST PRACTICE

Bedrock Guardrails Integration: Amazon Bedrock Guardrails provide a native output filtering layer that can be configured with Cedar policy decisions as context. When Cedar returns an obligation to redact PII, Bedrock Guardrails can apply the redaction pattern to the LLM output in a single integrated call. This eliminates the need to post-process raw LLM output in application code.